

Artículo – Redes Neuronales y Deep Learning

1. Resumen del modelo

1.1 Selección del modelo

Para el análisis práctico de la arquitectura descrita en el artículo, se ha empleado una implementación de ResUnet (Residual U-Net) localizada en un repositorio público de GitHub, perteneciente al usuario *rishikksh20*.

1.2 Configuración del Entorno

El primer paso consistió en la configuración de un entorno de trabajo en Python. Esto se logró ejecutando el siguiente comando para instalar todas las dependencias y librerías necesarias, las cuales estaban especificadas en el archivo requirements.txt del repositorio:

```
pip install -r requirements.txt
```

Adicionalmente, se requirió la instalación de la librería torchsummary, ya que el modelo está implementado en PyTorch. Esta librería proporciona una funcionalidad análoga al método .summary() de Keras, permitiendo visualizar un resumen detallado de la arquitectura, las formas (shapes) de salida de cada capa y el número de parámetros:

```
pip install torchsummary
```

Una vez configurado el entorno, se procedió a instanciar el modelo. Se utilizó el siguiente script de Python para importar la arquitectura, definir la forma del tensor de entrada y generar el resumen:

```
from core.res_unet import ResUnet
```

```
from torchsummary import summary
```

```
INPUT_SHAPE = (3, 224, 224)
```

```
model = ResUnet(INPUT_SHAPE[0])
```

```
summary(model, INPUT_SHAPE)
```

La ejecución de este script arroja el siguiente resumen, que será la base para el análisis de la arquitectura y el número de parámetros.

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 224, 224]	1,792
BatchNorm2d-2	[-1, 64, 224, 224]	128
ReLU-3	[-1, 64, 224, 224]	0
Conv2d-4	[-1, 64, 224, 224]	36,928
Conv2d-5	[-1, 64, 224, 224]	1,792
BatchNorm2d-6	[-1, 64, 224, 224]	128
ReLU-7	[-1, 64, 224, 224]	0
Conv2d-8	[-1, 128, 112, 112]	73,856
BatchNorm2d-9	[-1, 128, 112, 112]	256
ReLU-10	[-1, 128, 112, 112]	0
Conv2d-11	[-1, 128, 112, 112]	147,584
Conv2d-12	[-1, 128, 112, 112]	73,856
BatchNorm2d-13	[-1, 128, 112, 112]	256
ResidualConv-14	[-1, 128, 112, 112]	0
BatchNorm2d-15	[-1, 128, 112, 112]	256
ReLU-16	[-1, 128, 112, 112]	0
Conv2d-17	[-1, 256, 56, 56]	295,168
BatchNorm2d-18	[-1, 256, 56, 56]	512
ReLU-19	[-1, 256, 56, 56]	0
Conv2d-20	[-1, 256, 56, 56]	590,080
Conv2d-21	[-1, 256, 56, 56]	295,168
BatchNorm2d-22	[-1, 256, 56, 56]	512

...
Forward/backward pass size (MB): 1087.95
Params size (MB): 49.76
Estimated Total Size (MB): 1138.28

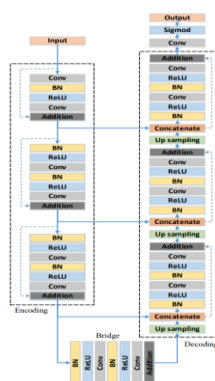


Ilustración 1. Comparativa Output (izquierda) y arquitectura propuesta ResUnet (derecha)

TABLE I
THE NETWORK STRUCTURE OF RESUNET.

	Unit level	Conv layer	Filter	Stride	Output size
Input					$224 \times 224 \times 3$
Encoding	Level 1	Conv 1	$3 \times 3 / 64$	1	$224 \times 224 \times 64$
		Conv 2	$3 \times 3 / 64$	1	$224 \times 224 \times 64$
	Level 2	Conv 3	$3 \times 3 / 128$	2	$112 \times 112 \times 128$
		Conv 4	$3 \times 3 / 128$	1	$112 \times 112 \times 128$
	Level 3	Conv 5	$3 \times 3 / 256$	2	$56 \times 56 \times 256$
		Conv 6	$3 \times 3 / 256$	1	$56 \times 56 \times 256$
Bridge	Level 4	Conv 7	$3 \times 3 / 512$	2	$28 \times 28 \times 512$
		Conv 8	$3 \times 3 / 512$	1	$28 \times 28 \times 512$
	Level 5	Conv 9	$3 \times 3 / 256$	1	$56 \times 56 \times 256$
		Conv 10	$3 \times 3 / 256$	1	$56 \times 56 \times 256$
	Level 6	Conv 11	$3 \times 3 / 128$	1	$112 \times 112 \times 128$
		Conv 12	$3 \times 3 / 128$	1	$112 \times 112 \times 128$
Decoding	Level 7	Conv 13	$3 \times 3 / 64$	1	$224 \times 224 \times 64$
		Conv 14	$3 \times 3 / 64$	1	$224 \times 224 \times 64$
Output		Conv 15	1×1	1	$224 \times 224 \times 1$

Tabla 1. Tabla extraída del paper

1.2 Comparación modelo original e implementado

Comparando nuestro modelo recién implementado con lo observado en el artículo y en la respectiva ilustración y tabla observamos que:

- Ruta de Codificación (Encoding): Se puede apreciar en el output como las dimensiones de la imagen se reducen mientras que los canales se van aumentando:
 - p.e: Conv2d-1 a Conv2d-8: Corresponden a los primeros bloques de la "Encoding path".
- Uso de unidades Residuales: En la capa ResidualConv-14. Esta es la implementación de la conexión Add que se muestra en la figura 1 del artículo. Esto confirma que la arquitectura cargada es una ResUnet.
- Ruta de Decodificación (Decoding): No se aprecia en la captura extraída de VisualStudio Code pero al final del output se ven capas de Unsampling como ConvTranspose2d.
- Conexiones de Salto (Skip Connections): Se observan operaciones de Concatenate que fusionan mapas de características de la ruta de codificación con la ruta de decodificación en los niveles correspondientes.

1.3 Número de parámetros

El artículo reporta que su implementación de ResUnet posee 7.8 millones de parámetros, lo cual contrasta favorablemente con los 30.6 millones de la arquitectura U-Net estándar.

El modelo analizado en este trabajo presenta un total de 13 millones de parámetros. Esta discrepancia no invalida la implementación, sino que se justifica como una variación en la "anchura" (width) del modelo. Mientras que el artículo describe una configuración específica de filtros (ej. 64, 128, 256, 512), la implementación de terceros seleccionada utiliza un número de filtros base superior, incrementando el conteo total de parámetros. El argumento central del artículo se mantiene intacto: la implementación analizada sigue siendo más de 2.3 veces más ligera que la U-Net original.

```

Total params: 13,043,009
Trainable params: 13,043,009
Non-trainable params: 0
-----
Input size (MB): 0.57
Forward/backward pass size (MB): 1087.95
Params size (MB): 49.76
Estimated Total Size (MB): 1138.28
-----

```

Ilustración 2. Parámetros modelo

2. Tensor de Entrada

Para que el modelo ResUnet pueda procesar una imagen, esta debe ser transformada en un tensor con un formato (shape) y una distribución de valores específicos.

2.1. Ejemplo y Pre-procesamiento

Como ejemplo de entrada, se utiliza una imagen aérea de una carretera (map). De acuerdo con la Tabla I del artículo, el modelo fue entrenado con imágenes de 224x224 píxeles. Por lo tanto, la imagen de ejemplo seleccionada se redimensiona a esta resolución.

2.2. Forma del Tensor (Shape)

La implementación del modelo (en PyTorch) espera un formato de tensor (N, C, H, W), donde:

- N (Batch Size): 1, ya que procesamos una sola imagen de ejemplo.
- C (Channels): 3, correspondientes a los canales de color RGB de la imagen².
- H (Height): 224, la altura de la imagen redimensionada.
- W (Width): 224, la anchura de la imagen redimensionada.

El tensor de entrada final para nuestro ejemplo tiene la forma: (1, 3, 224, 224).

2.3. Distribución de Valores

Las redes neuronales rara vez se entrenan con los valores de píxeles brutos (en el rango 0-255). Se aplica un paso de normalización dividiendo cada valor de píxel por 255.0.

Esto transforma la distribución de valores del tensor al rango [0.0, 1.0]. Esta normalización es una práctica estándar que asegura que todas las características de entrada tengan una escala similar, lo que resulta en una convergencia más rápida y estable durante el entrenamiento del modelo.



Imagen de Entrada (224x224, Normalizada)

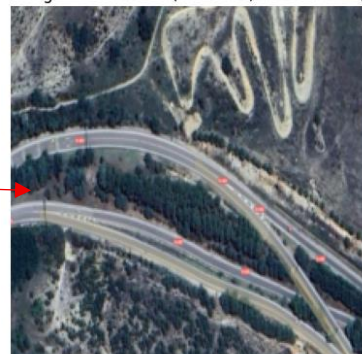


Ilustración 3. Ejemplo transformación imagen

3. Logits de salida

3.1. Justificación de la Elección

La arquitectura ResUnet descrita en el artículo es una Red Neuronal Convolutiva (CNN) diseñada para segmentación semántica. Los componentes positional encoding y mask

attention layer son elementos característicos de arquitecturas *Transformer*, las cuales no forman parte del diseño de este modelo.

Por lo tanto, el único componente de salida analizable y relevante para esta arquitectura son los logits de salida.

3.2. Función de los Logits

En el contexto de este modelo de segmentación binaria (carretera vs. no carretera), los logits son las puntuaciones brutas generadas por la última capa convolucional de la red.

La función de este tensor de logits, que para nuestro ejemplo tiene la forma (1, 1, 224, 224), es asignar un valor numérico a cada píxel de la imagen de entrada. Este valor representa la confianza de la red en que dicho píxel pertenece a la clase "carretera".

3.3. Activaciones Relevantes y Valores

Las activaciones más relevantes del modelo se encuentran en este tensor de logits:

- Valores Positivos Altos: Indican una alta confianza del modelo en que el píxel es "carretera".
- Valores Negativos Altos: Indican una alta confianza en que el píxel es "no carretera" (fondo).
- Valores Cercanos a Cero: Representan incertidumbre o el límite de decisión del modelo.

Posteriormente, estos logits son pasados a través de una función de activación Sigmoid, que "aplata" estos valores al rango [0.0, 1.0], convirtiéndolos en el mapa de probabilidad final.

Para visualizar las activaciones más relevantes y la confianza subyacente del modelo, se debe analizar el *heatmap* del tensor de logits. Las zonas con valores más altos corresponden a las áreas donde el modelo identifica con mayor certeza la presencia de carreteras, tal como aprendió durante su entrenamiento.

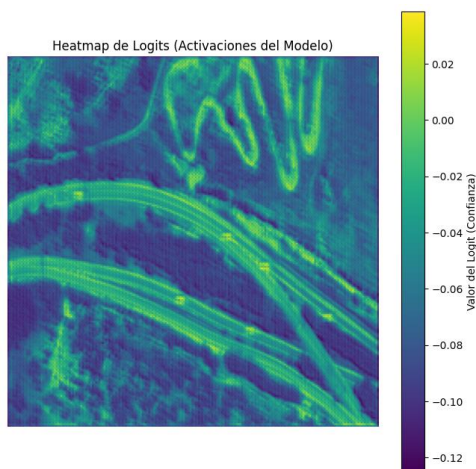


Ilustración 4. Heatpmap de Logits

4. Referencias

[LINK AL GITHUB](#)

Zhang, Z., Liu, Q., & Wang, Y. (2017). *Road Extraction by Deep Residual U-Net*. ArXiv.

NOTA I: Añadir que al haber complementado el artículo con imágenes, este sobrepasa las 4 páginas de longitud, pero el contenido escrito del mismo si que se ciñe a lo solicitado

5. Anexo código

En este anexo se incluye el código empleado para realizar el apartado 2 y 3 el artículo.

TRANSFORMACIÓN IMAGEN

```
import torch

from PIL import Image

import torchvision.transforms as T

import matplotlib.pyplot as plt

import numpy as np


# --- Define las transformaciones ---

transform = T.Compose([

    T.Resize((224, 224)), # Redimensionar a 224x224

    T.ToTensor(),      # Convertir a Tensor (y pone valores en [0.0, 1.0])

    # T.ToTensor() ya divide por 255.0, así que la normalización [0,1] está hecha.

])


# --- Carga imagen de ejemplo ---

image_path = 'map.png'


try:

    # Abre la imagen

    img = Image.open(image_path).convert('RGB') # Asegura que sea RGB (3 canales)


# --- Aplica las transformaciones ---
```

```
# Crea un tensor (C, H, W) -> (3, 224, 224)

tensor = transform(img)

# --- Añade la dimensión del "batch" (N) ---

# (N, C, H, W) -> (1, 3, 224, 224)

input_tensor = tensor.unsqueeze(0)

# --- Imprime la información requerida---

print(f"Forma (shape) del tensor de entrada: {input_tensor.shape}")

print(f"Distribución: Valor mínimo: {input_tensor.min()}")

print(f"Distribución: Valor máximo: {input_tensor.max()}")

print("-----")

# --- Visualiza la imagen que el modelo "verá" ---

img_to_show = tensor.permute(1, 2, 0).numpy()

plt.imshow(img_to_show)

plt.title("Imagen de Entrada (224x224, Normalizada)")

plt.axis('off') # Oculta los ejes

plt.show()

except FileNotFoundError:

    print(f"Error: No se encontró la imagen en '{map.png}'.")

    print("Por favor, descarga una imagen aérea de una carretera y actualiza la ruta.")
```

CREACIÓN DEL HEATMAP DE LOGITS

```
import torch

import torchvision.transforms as T

from PIL import Image

import matplotlib.pyplot as plt

import numpy as np

from core.res_unet import ResUnet
```

```
# Carga la arquitectura del modelo

model = ResUnet(3)

model.eval() # Pone el modelo en modo de evaluación (desactiva dropout, etc.)


# Carga y prepara el tensor de entrada

transform = T.Compose([
    T.Resize((224, 224)),
    T.ToTensor(),
])

try:
    img = Image.open('map.png').convert('RGB')
    input_tensor = transform(img).unsqueeze(0) # Shape: (1, 3, 224, 224)
except FileNotFoundError:
    print("Error: No existe la imagen.")
    # Creando un tensor de 'dummy' para que el script corra
    input_tensor = torch.rand(1, 3, 224, 224)


# --- OBTENER LOS LOGITS ---

# Pasa la imagen por el modelo

with torch.no_grad(): # Desactiva el cálculo de gradientes
    # 'output' es lo que sale del modelo.
    # Asumimos que es la probabilidad (post-sigmoid) [0.0, 1.0]
    output_prob = model(input_tensor)


# Calcula los Logits (la inversa de la Sigmoid)

# logit = log(p / (1 - p))

# Usamos 'clamp' para evitar log(0) si la salida es 0.0 o 1.0
epsilon = 1e-7 # Un valor muy pequeño para estabilidad numérica
output_prob_clamped = torch.clamp(output_prob, epsilon, 1 - epsilon)
```

```
logits = torch.log(output_prob_clamped / (1 - output_prob_clamped))
```

```
# Muestra los valores de los logits
```

```
print(f"Forma (shape) del tensor de logits: {logits.shape}")
```

```
print(f"Valor mínimo de los logits: {logits.min().item()}")
```

```
print(f"Valor máximo de los logits: {logits.max().item()}")
```

```
print("-----")
```

```
# .squeeze() -> Quita las dimensiones de (1, 1, 224, 224) a (224, 224)
```

```
# .detach() -> Separa el tensor del grafo de cómputo
```

```
# .cpu() -> Mueve el tensor de la GPU (si la tuvieras) a la CPU
```

```
# .numpy() -> Convierte a formato numpy para matplotlib
```

```
logits_image = logits.squeeze().detach().cpu().numpy()
```

```
plt.figure(figsize=(8, 8))
```

```
plt.imshow(logits_image, cmap='viridis') # 'viridis' es un buen heatmap
```

```
plt.title("Heatmap de Logits (Activaciones del Modelo)")
```

```
plt.colorbar(label="Valor del Logit (Confianza)")
```

```
plt.axis('off')
```

```
plt.show()
```